



Cofinanciado por
la Unión Europea



GOBIERNO DE ESPAÑA
MINISTERIO DE INDUSTRIA
COMERCIO Y TURISMO
EOI Escuela de
organización
industrial



Fondos Europeos



PYTHON PARA IA

ANÁLISIS DE DATOS API DIGIMON

ÍNDICE

1. INTRODUCCIÓN	3
2. DESCRIPCIÓN DEL PROYECTO	3
3. TECNOLOGÍAS UTILIZADAS	3
4. ESTRUCTURA DEL CÓDIGO	4
4.1 MAIN.PY	4
4.2 DIGIMON.PY	4
4.3 INDEX.HTML	4
4.4 SCRIPT.JS	5
5. FUNCIONAMIENTO	5
6. CAPTURAS DE PANTALLA	6
7. CONSIDERACIONES ADICIONALES	7
8. BIBLIOGRAFÍA	8
9. CONCLUSIONES	9

1. INTRODUCCIÓN

Con este proyecto se pretende realizar una aplicación web que acceda a una API externa, en este caso relacionada con la franquicia Digimon, para poder obtener los datos de estos y realizar un análisis en función del tipo y del nivel al que pertenecen los digimons y también poder generar una gráfica para poder cuantificarlo. El usuario podrá seleccionar en un pequeño select en la parte del front el número de digimons que quiere analizar, empezando por el primero.

2. DESCRIPCIÓN DEL PROYECTO

El proyecto constará de la parte del front, donde el usuario podrá interactuar para seleccionar el número de digimons a analizar y la parte del back o servidor, donde se ubicarán las rutas y las funciones necesarias para poder acceder a la API y poder manejar todos los datos necesarios para poder enviarlos a la parte de front.

3. TECNOLOGÍAS UTILIZADAS

Las tecnologías utilizadas son las siguientes:

- Python y Flask: Para backend y servidor web.
- JS: Para gestionar la conexión con back y funciones relacionadas con el front.
- HTML: Para la interfaz que gestionará y permitirá la visualización al usuario.
- CSS y Bootstrap: Para mejorar el diseño y aspecto visual del HTML.

4. ESTRUCTURA DEL CÓDIGO

4.1 MAIN.PY

Este archivo contiene toda la configuración relacionada con Flask y con la gestión de las rutas.

Contiene tres rutas:

1. **/digimon**. Realiza la petición a la url de la API y obtiene la info del número predeterminado por defecto o en su caso del número pasado por parámetro y los devuelve en objeto JSON.
2. **/digimon/grafica**. Realiza la misma función que la url anterior, pero en lugar de devolver un objeto JSON devuelve una gráfica en formato imagen con los datos analizados por atributo
3. **/digimon/graficanivel**. Realiza la misma función que /digimon/gráfica, pero en este caso devuelve la gráfica con el análisis en función del nivel de los digimons.

4.2 DIGIMON.PY

Este archivo contiene las siguientes funciones:

1. **obtener_datos_digimon(digimon_id)**. Recibe un número y realiza la petición URL a la API del número de digimon pasado por parámetro.
2. **cargar_todos_digimon(num_digimon)**. Recibe el número de digimon los cuales se van a analizar para posteriormente recorrer mediante un for llamando a la función anterior y obtener todos los digimon y meterlos en una lista. Finalmente se devuelve un objeto de tipo DataFrame.
3. **analizar_datos(df)**. Calcula la media de la columna "Lanzamiento" y obtiene la distribución de frecuencias de los valores en las columnas "Atributo" y "Nivel" de un DataFrame. Devuelve un diccionario con estos análisis junto con las distribuciones originales en formato Series.

4.3 INDEX.HTML

Este archivo se encarga de enlazar con el archivo script.js y para definir la estructura básica de la interfaz.

Permite al usuario seleccionar el número de digimons a analizar y poder ver las gráficas, media de año de lanzamiento y la info específica de cada uno de los digimons mediante los elementos Cards de Bootstrap.

4.4 SCRIPT.JS

1. **crearSelect ()**: Crea un menú desplegable (select) con opciones que van de 5 a 50, incrementándose en pasos de 5. Cuando se selecciona una opción, se llama a la función `mostrarTodosDigimons ()` para actualizar la cantidad de Digimons mostrados.

2. **mostrarAnálisisDatos (cantidad = NUM_DIGIMONS)**: Genera el HTML para mostrar imágenes de gráficos relacionados con los Digimons y la media de lanzamiento de estos. Hace una petición fetch a una API local (en Python) para obtener datos analíticos de los Digimons y los muestra en la página.

3. **mostrarTodosDigimons(cantidad = NUM_DIGIMONS)**: Muestra un spinner de carga mientras se obtienen los datos. Llama a `mostrarAnálisisDatos ()` para mostrar gráficos y la media de lanzamiento de los Digimons.

Luego, realiza peticiones a una API externa (`digi-api.com`) para obtener los datos individuales de cada Digimon, y llama a `mostrarDigimon()` para mostrar los detalles. Oculta el spinner de carga una vez se han obtenido y mostrado todos los Digimons.

4. **mostrarDigimon (i, digimon)**: Crea el HTML para mostrar la tarjeta de un Digimon específico, incluyendo su nombre, nivel, descripción, atributo, imagen, y campos asociados (si están disponibles).

5. **inicializar ()**: Inicializa la aplicación llamando a `crearSelect()` para construir el menú desplegable y a `mostrarTodosDigimons()` para cargar los primeros Digimons por defecto.

5. FUNCIONAMIENTO

El usuario tiene un desplegable con un select que irá aumentando de 5 en 5 progresivamente, que serán los digimons que la aplicación analizará.

Se enviará el número y el front conectará con el servidor de Flask, que realizará la petición a la URL de la API, la cual obtendrá los datos necesarios y los analizará.

El servidor Flask mediante las rutas enviará los datos analizados al front en forma de dos gráficas, por nivel y por atributo.

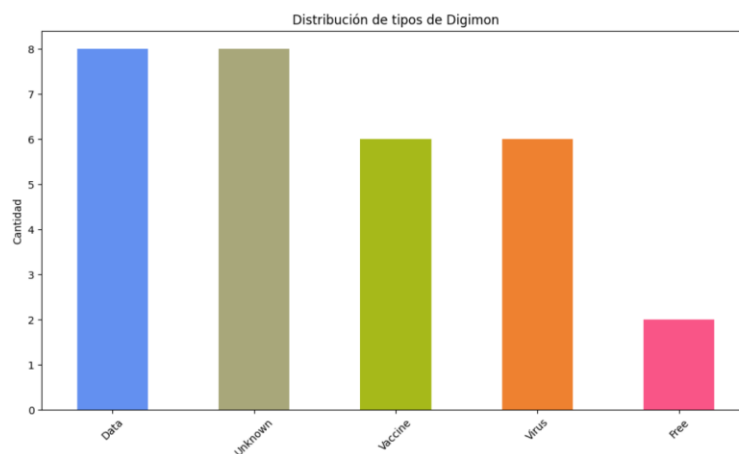
El usuario con la interfaz web podrá visualizar las dos gráficas, así como una información detallada de cada uno de los digimon que ha seleccionado previamente.

6. CAPTURAS DE PANTALLA

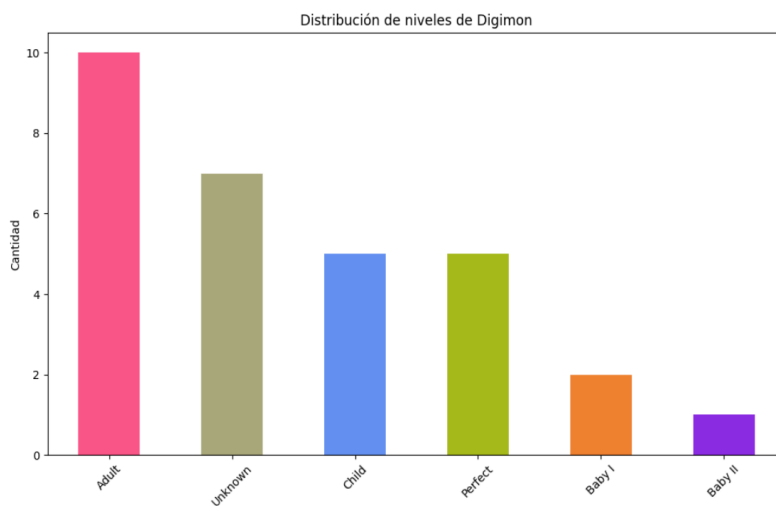
DIGIMONS

Selecciona la cantidad de Digimons:

30



1 Gráfica atributos



2 Gráfica niveles

Media de lanzamiento 1997

1. Agumon

Nivel: Child

Descripción: A Reptile Digimon with an appearance resembling a small dinosaur, it has grown and become able to walk on two legs. Its strength is weak as it is still in the process of growing, but it has a fearless and rather ferocious personality. Hard, sharp claws grow from both its hands and feet, and their power is displayed in battle. It also foreshadows an evolution into a great and powerful Digimon. Its Special Move is spitting a fiery breath from its mouth to attack the opponent (Baby Flame).

Atributo: Vaccine



3 Media lanzamiento e info Digimon.

7. CONSIDERACIONES ADICIONALES

- **Seguridad:** La aplicación utiliza una API que debe ocultarse para que cualquiera no pueda acceder a ella.
- **Diseño:** La aplicación podría tener un diseño mejor, en función del nivel o el tipo tener unas clases o estilos diferentes.
- **Rendimiento:** Sería interesante realizar una opción en la parte del front para que el usuario también pueda filtrar en función de los niveles o de los atributos.

8. BIBLIOGRAFÍA

- DAPI:
<https://digi-api.com/>
- Flask:
<https://flask.palletsprojects.com/en/3.0.x/>
- Python:
<https://docs.python.org/3/tutorial/>
- JS:
<https://developer.mozilla.org/es/docs/Web/JavaScript>
- CSS:
<https://developer.mozilla.org/es/docs/Web/CSS>
- Bootstrap:
<https://getbootstrap.com/>
- Matplotlib:
<https://matplotlib.org/>
- Pandas:
<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>
- CORS:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

9. CONCLUSIONES

Con este proyecto se ha podido ver lo sencillo que puede ser realizar análisis y procesamiento de datos a través de una API totalmente gratuita y con un acceso sencillo a herramientas y software en internet, fundamentalmente Python junto con Flask.

Las principales razones de la utilización de Python junto a Flask son:

1. Facilidad de Uso y Aprendizaje

Python Es conocido por su sintaxis clara y legible, lo que facilita la escritura y el mantenimiento del código. Flask es un framework minimalista, ideal para principiantes, ya que no impone una estructura rígida.

2. Flexibilidad y Control

Flask ofrece más flexibilidad y control sobre la estructura de la aplicación. Puedes seleccionar los componentes que necesitas.

3. Escalabilidad

Flask aunque es ligero, es lo suficientemente potente para manejar aplicaciones complejas. Flask permite comenzar con algo pequeño y sencillo y luego escalar la aplicación con facilidad. Puedes integrar extensiones y herramientas según sea necesario.

4. Gran Ecosistema y Extensiones

Python tiene una amplia variedad de bibliotecas y módulos disponibles para casi cualquier tarea. Estas bibliotecas pueden integrarse fácilmente con Flask. Flask ofrece una gran cantidad de extensiones para características como autenticación, manejo de formularios, bases de datos, y más.

5. Desarrollo Rápido

- Ambos permiten un desarrollo ágil. Flask tiene un enfoque de configuración mínima, lo que significa que puedes comenzar a desarrollar tu aplicación rápidamente sin tener que configurar una gran cantidad de archivos o configuraciones.